

Avoid Enemies and Gaps in 3Dplatformer Game via DQN

113221023 陳宗毅

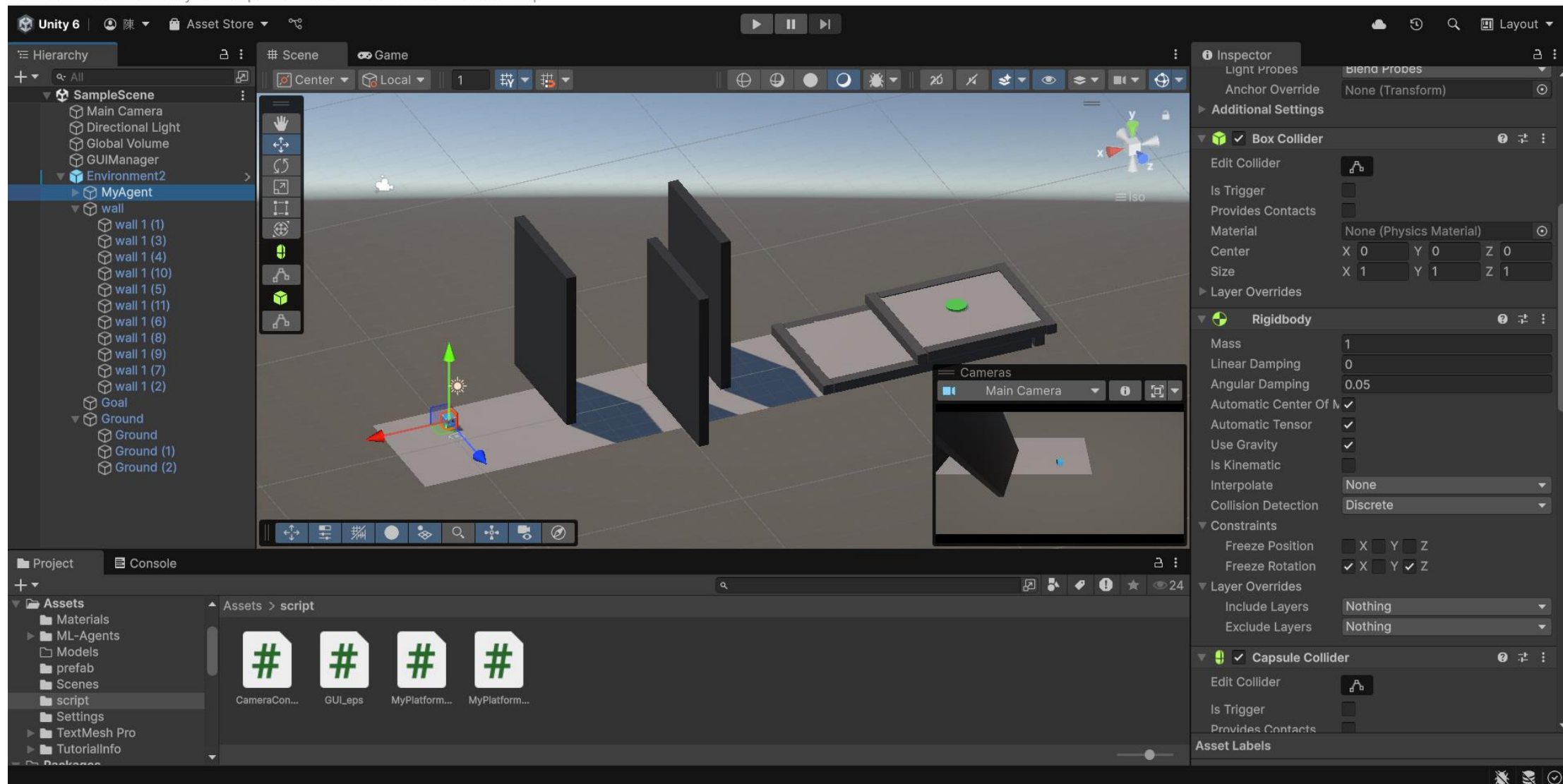
Contents

1. Simulation Environment(P3)
2. Objectives(P7)
3. Agent(P8)
4. DQN(P14)
5. Training Process(P19)
6. Progress and Future Work(P20)

Simulation Environment

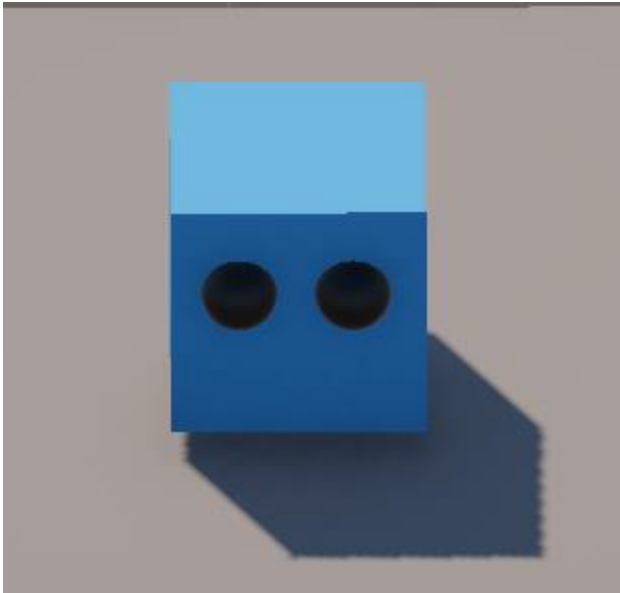
Unity is a powerful real-time 3D development platform widely used for building interactive simulations, games, and virtual environments.

Characters can Navigate complex 3D worlds and Interact with dynamic objects and physics.

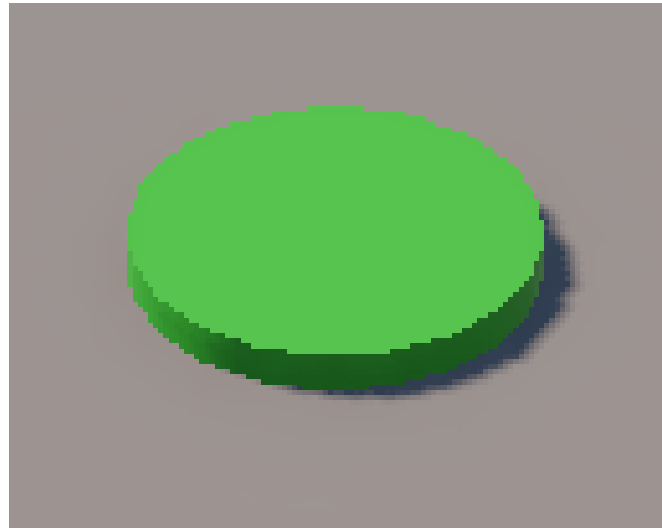


Simulation Environment

Characters:



Agent



Goal



Enemy

Simulation Environment

In this project, we need some tools as well.

- **Python:**
 - Numpy
 - Pytorch
 - yaml, argparse,
- **Unity:**
 - ml-agents

Objectives

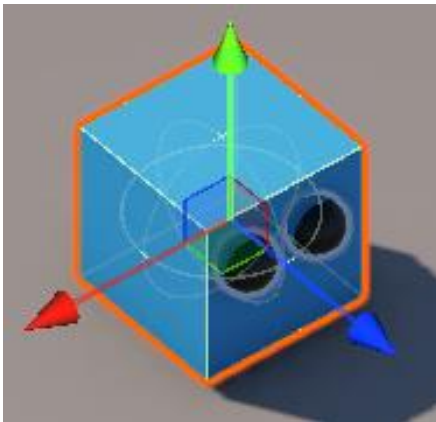
Train an autonomous agent with DQN in Unity ML-Agents that can:

- Reach the goal
- Avoid gaps
- Bypass wall
- Jump up stairs
- Avoid enemies

Agent (action)

Infos:

- Position(x,y,z)
- Facing direction(Quaternion)
- Only rotate around the Y-axis



Actions:

- Idle
- Move Forward
- Turn Left
- Turn Right
- Jump Forward(On ground only)

Agent (Reward)

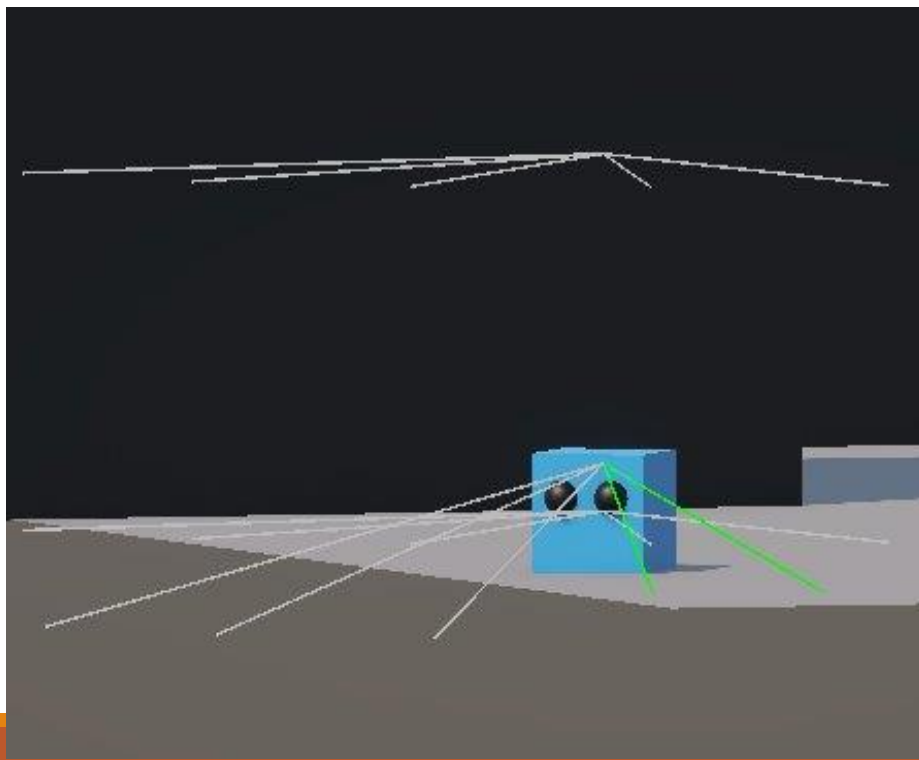
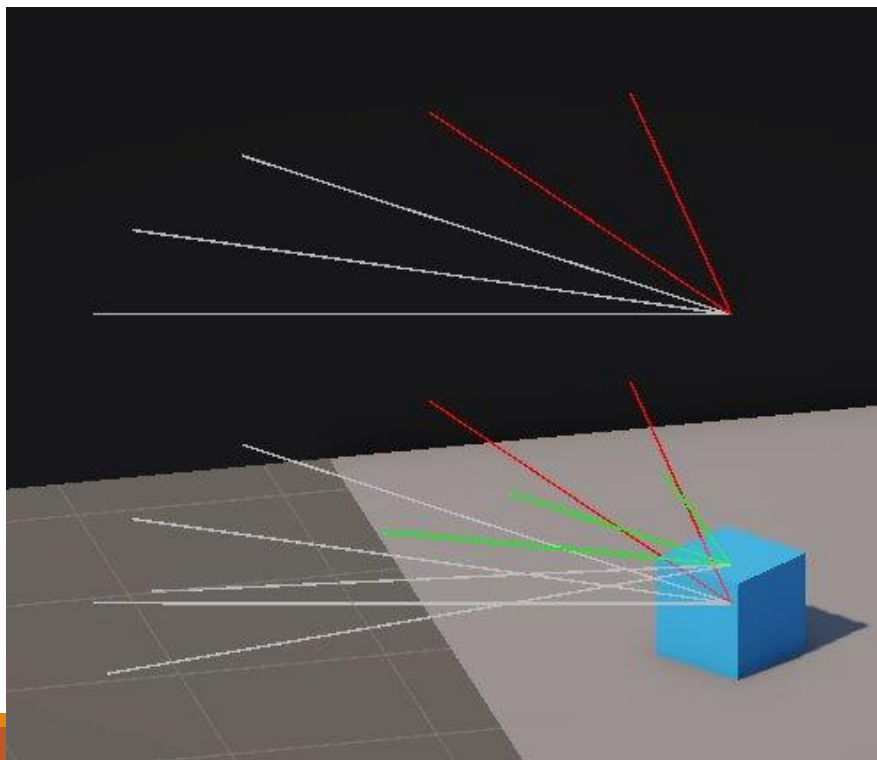
Reward:

- Take step: -0.001
- Reach the goal: +10.0
- Fall off the platform ($Y < -10$): -10.0
- Touch an enemy: -10.0

It is terminal as agent out of steps, reach goal or being killed.

Agent (Ray)

We use rays to help agent to detect wall, ground, gaps and enemies.



Agent (Observation)

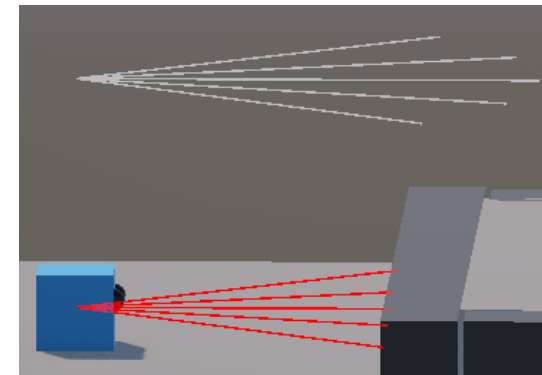
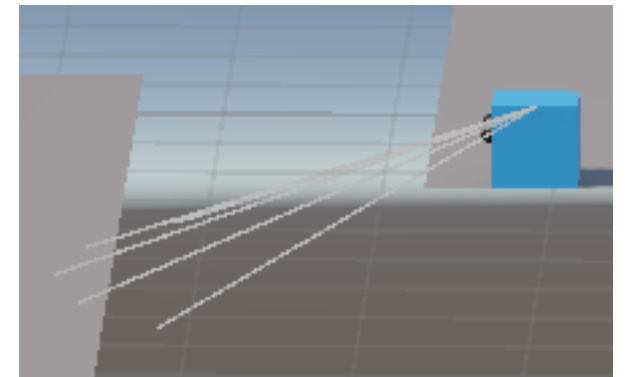
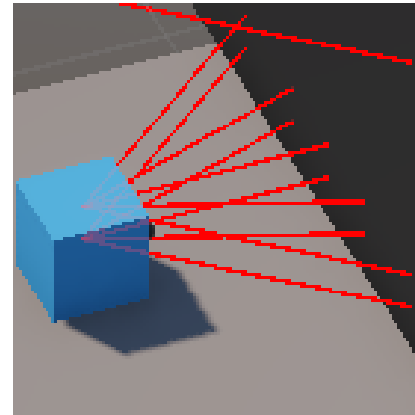
Observations(1/2):

- Horizontal/Vertical Distance to Goal
- Angle Between Agent and Goal
- Local Velocity Magnitude
- Is Grounded?

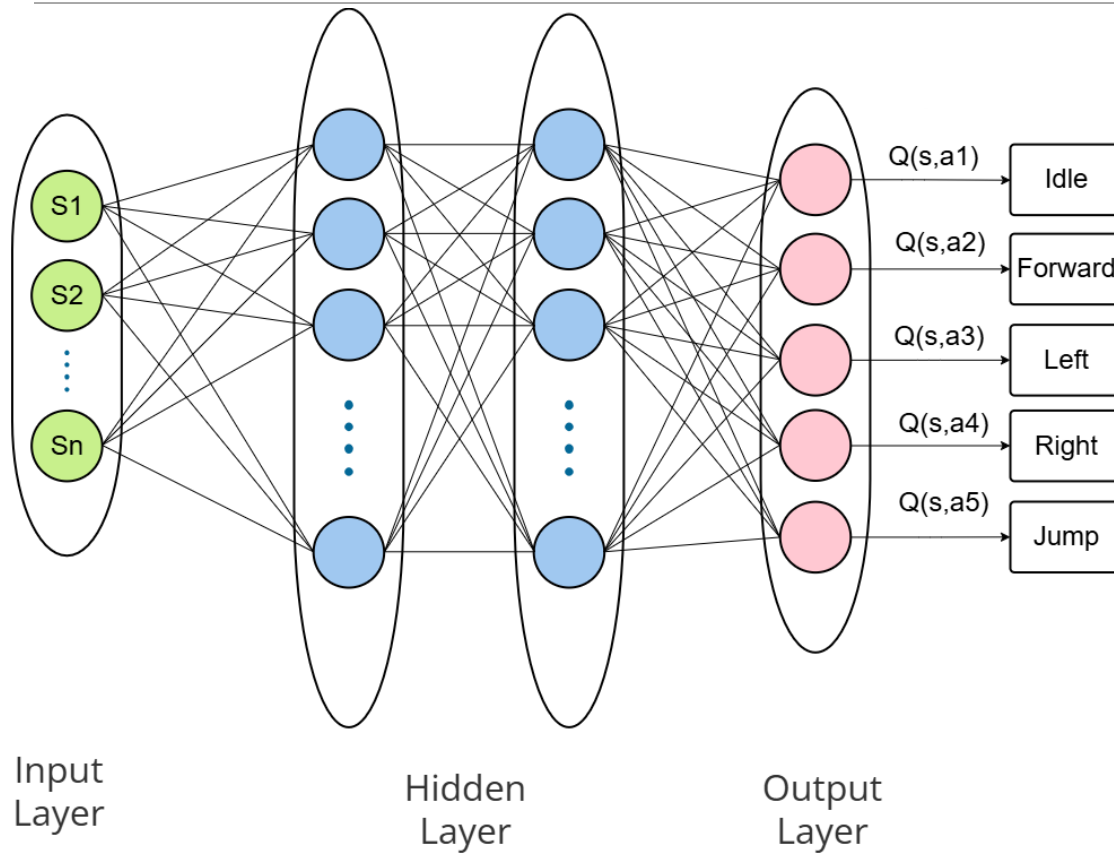
Agent (Observation)

Observations(2/2):

- Three Types of Middle Ray Distances
- Wall Directly in Front
- Gap Detected Forward
- Climbable/Downward Surface Ahead
- Wall/Enemies Within Front Angular Range
- Very Close Obstacle Ahead



DQN



- **Bellman Equation of Q-Learning**

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- **MSE Loss function of DQN**

$$\mathcal{L}(\theta) = \left[r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-) - Q(s, a; \theta) \right]^2$$

DQN (Parameters)

model:

hidden_sizes: [128,128]

gamma: 0.95

lr: 0.0005

buffer_size: 10000

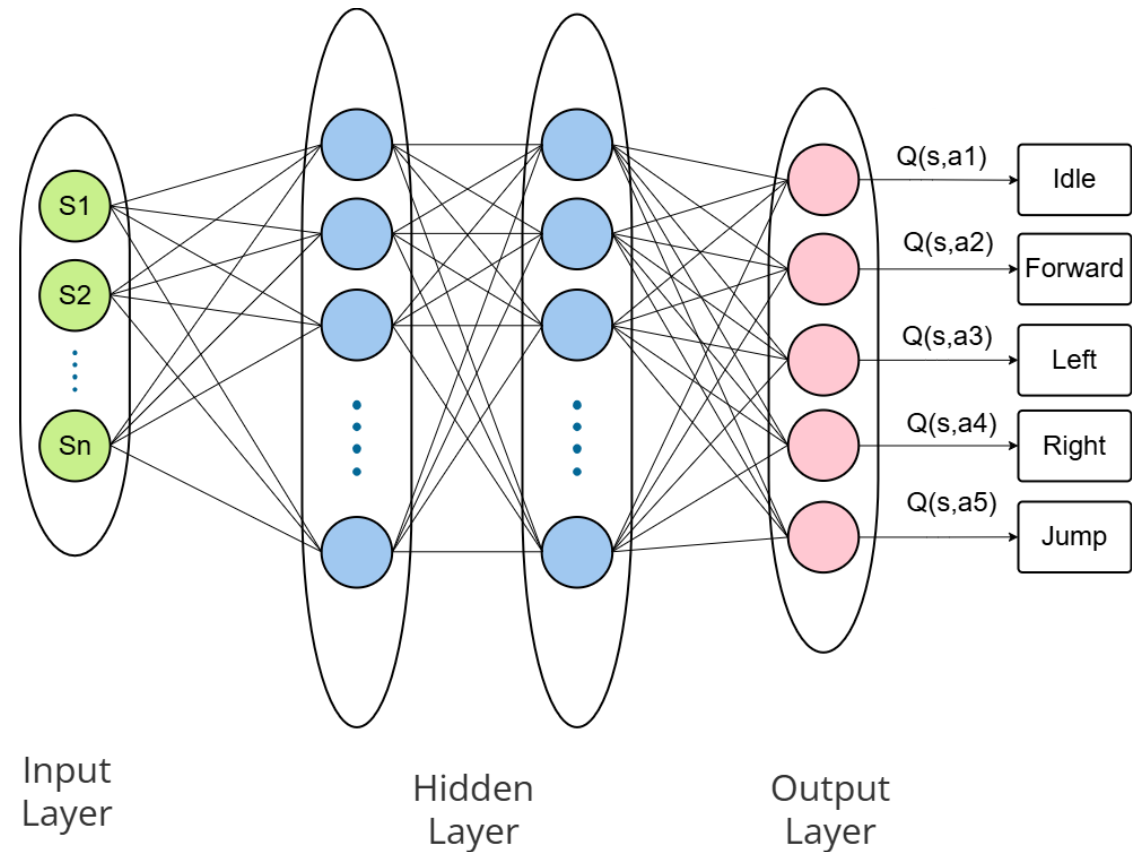
batch_size: 64

epsilon_start: 1.0

epsilon_end: 0.01

epsilon_decay: 0.999

target_update_freq: 100



DQN (Parameters)

model:

hidden_sizes: [128,128]

gamma: 0.95

lr: 0.0005

buffer_size: 10000

batch_size: 64

epsilon_start: 1.0

epsilon_end: 0.01

epsilon_decay: 0.999

target_update_freq: 100

- **Bellman Equation of Q-Learning**

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- **Transitions:** (s, a, s', r, done)

- **Loss function of DQN**

$$\mathcal{L}(\theta) = \left[r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-) - Q(s, a; \theta) \right]^2$$

DQN (Parameters)

model:

hidden_sizes: [128,128]

gamma: 0.95

lr: 0.0005

buffer_size: 10000

batch_size: 64

epsilon_start: 1.0

epsilon_end: 0.01

epsilon_decay: 0.999

target_update_freq: 100

Exploration VS Exploitation

$$\epsilon = \max(0.999^{step}, 0.1)$$

DQN (Parameters)

model:

hidden_sizes: [128,128]

gamma: 0.95

lr: 0.0005

buffer_size: 10000

batch_size: 64

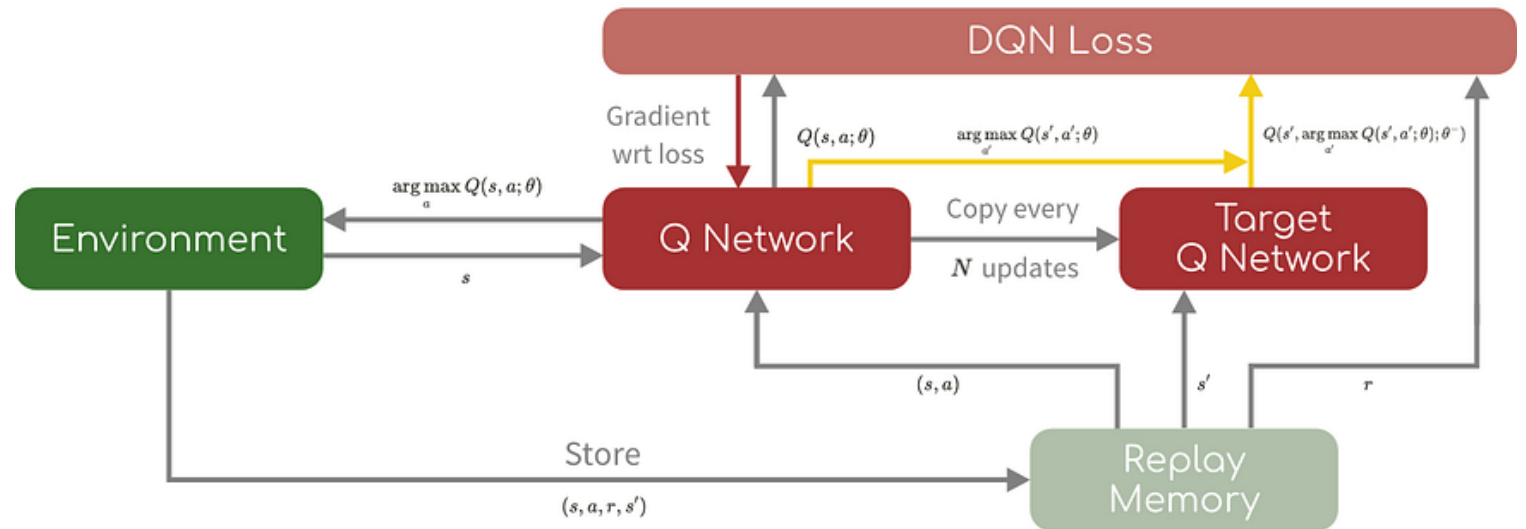
epsilon_start: 1.0

epsilon_end: 0.01

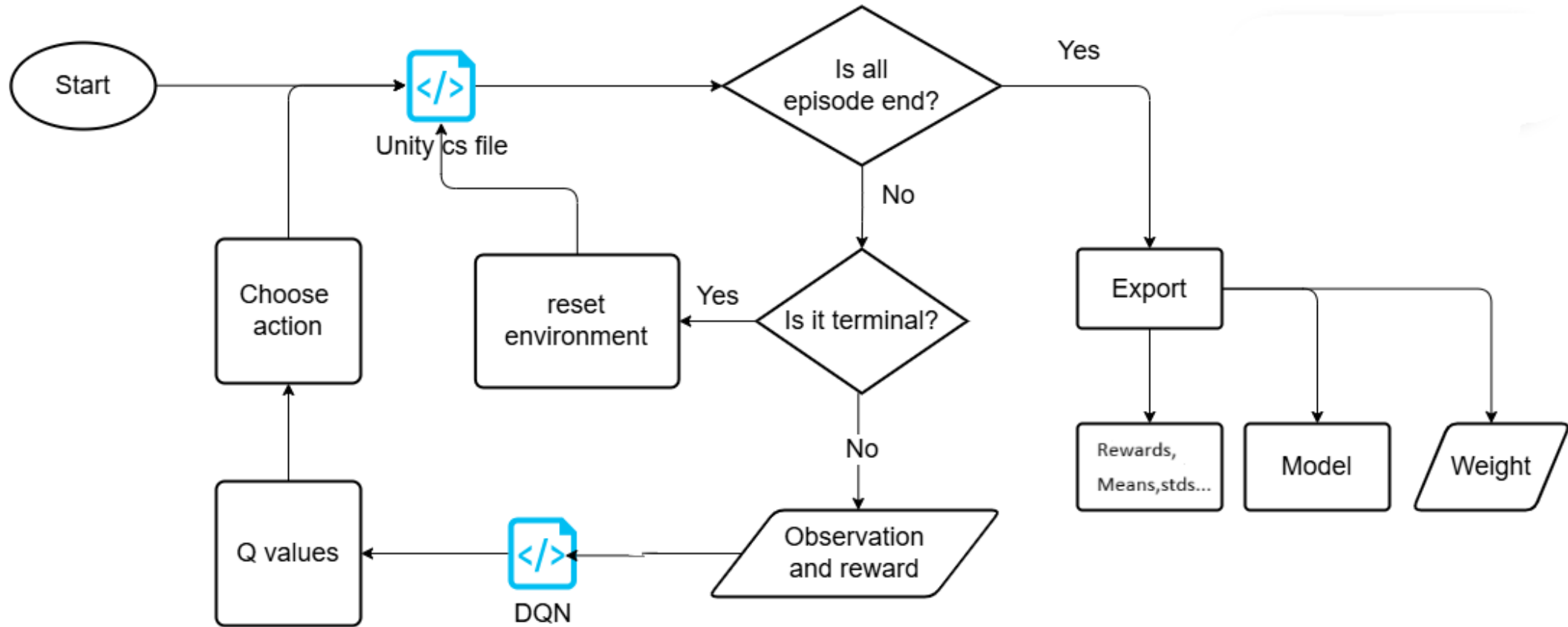
epsilon_decay: 0.999

target_update_freq: 100

$$\mathcal{L}(\theta) = \left[r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta^-) - Q(s, a; \theta) \right]^2$$



Training Process



Progress and Future Work

Progress: Go to randomly spawned goal, Avoid gaps, jump up stairs, bypass wall.

Future work: Avoid enemies, complete mixed environment, export datas for final report in figures.

Thanks for Listening!